

Assignment 1

Guidelines:

1. Logistics. Your solution must be submitted as a *typed* PDF document. Scanned handwritten solutions, solutions in any other format, or unreadable solutions will not be accepted. You are encouraged to use L^AT_EX.

Submit your solution in Crowdmark by **Tuesday, Feb 25, 23:59**. You are strongly encouraged to submit the solution at least *several days in advance* – last-minute technical problems in your systems and/or in Crowdmark systems will not grant any extension to the deadline.

2. Answering questions. In all assignments and exams in this course, unless specifically and explicitly stated otherwise:

- Establish correctness for all of your answers using rigorous proofs. Your solution will be marked based on its completeness and correctness as well as on the correctness, clarity and precision of your proofs.
- Describe algorithms in high-level pseudocode (at the same level of abstraction used in class, e.g. for dynamic programming). Do not provide low-level executable code. Also, if your code is overly abstract and does not sufficiently specify relevant parts (e.g., it is not obvious how to efficiently implement parts), you may lose marks.
- When you are asked to prove an upper bound on the running time of an algorithm, the intention is to prove the tightest upper bound that you can. Non-tight upper bounds may receive partial (or no) marks.

3. Academic integrity. Assignments are to be solved individually, and according to the course integrity policy (outlined in the syllabus) and the Code of Behaviour on Academic Matters. Here is a reminder of one of the relevant parts from the syllabus:

Large assignments should be solved individually, and written by yourself, without imitating online texts and without making use of generative AI (or any other software or automated tool). Any submitted assignment that is suspiciously similar to another assignment or to online texts will be automatically investigated as an integrity violation.

You are encouraged to ask questions on Piazza, without directly asking how to solve a problem or revealing the solution. You may not consult any other resources except your class notes, your textbook, Piazza, and assigned readings. Consulting any other resource or collaborating with other students is a violation of the academic integrity policy.

Question 1. In class we saw a greedy strategy for solving the interval scheduling problem. Describe a different greedy strategy that does *not* correctly solve this problem, and prove that for any integer n there is an input (i.e., a set of n intervals) for which the strategy produces an incorrect output. (5pts.)

(Note: Your answer must explicitly present an input for any $n \in \mathbb{N}$, and prove that the algorithm is incorrect on this input.)

Question 2. Given an n -vertex undirected graph $G = (V, E)$, a vertex cover is a set of vertices $S \subseteq V$ such that every edge $e \in E$ touches some $s \in S$. Our goal is to find a vertex cover of minimal size. Consider the following greedy algorithm:

```

let  $S = \emptyset, E' = E$ 
while  $E' \neq \emptyset$ :
    let  $e = (u, v)$  be an arbitrary edge in  $E'$ 
    add  $u$  and  $v$  to  $S$ 
    remove from  $E'$  all edges incident to  $u$  and to  $v$ 
return  $S$ 

```

Let $OPT(G)$ be the size of the minimal vertex cover for a graph G . Either prove that for every graph G , the greedy algorithm above outputs a vertex cover of size at most $2 \cdot OPT(G)$; or show that for every integer n there is a graph G on which this algorithm outputs a vertex cover of size more than $2 \cdot OPT(G)$. (10pts.)

Question 3. Consider an $n \times n$ matrix. A generalized diagonal is a sequence of n distinct indices $j_1, \dots, j_n \in [n]$ (i.e., $j_i \neq j_{i'}$ for all $i \neq i'$) such that in each row $i \in [n]$, the entry at column j_i is 1.¹

Given an $n \times n$ matrix, we want to find a generalized diagonal. Consider the following greedy algorithm:

```

for  $i = 1, \dots, n$ :
    let  $j_i$  be the left-most index such that the  $j_i^{th}$  entry in the  $i^{th}$  row is 1
    and  $j_i \neq j_k$  for all  $k \in \{0, \dots, i-1\}$ , or return "fail" if no such  $j_i$  exists
return  $j_1, \dots, j_n$ 

```

Prove that this algorithm is correct, or that for every integer n there is an input matrix on which this algorithm fails. (5pts.)

¹To develop intuition, Note that when the matrix has 1's across its standard diagonal, we can take $j_i = i$ for all $i \in [n]$, in which case the generalized diagonal is just the standard diagonal. Also note that some matrices don't have a generalized diagonal, for example the all-0's matrix.

Question 4. An LLM suggested an algorithm to multiply two integers, each represented by n bits, using recursion. The algorithm makes twelve recursive calls to multiplication problems on integers with $n/5$ bits, and then takes $O(n^{3/2})$ time to combine the results. The LLM did not provide further details, but claimed that this is the fastest known algorithm solving the problem.

What is the running time of the algorithm? Justify your answer using the Master theorem.² Is this algorithm faster than Karatsuba's algorithm? Is it faster than the best known algorithm for this problem? (5pts.)

Question 5. Prove that the Fast Fourier Transform algorithm runs in time $O(n \cdot \log n)$ on every integer $n \geq 2$ that is a power of two. The proof should use induction, and not use the Master theorem. (10pts.)

Question 6. An $n \times n$ matrix A is a Toeplitz matrix if for each diagonal of A , all entries along the diagonal have the same value. Here is a visual depiction:

$$\begin{pmatrix} a_{n-1} & a_{n-2} & a_{n-3} & \dots & a_1 & a_0 \\ a_n & a_{n-1} & a_{n-2} & \dots & a_2 & a_1 \\ \dots & \dots & \dots & \dots & \dots & \dots \\ \dots & \dots & \dots & \dots & \dots & \dots \\ a_{2n-3} & \dots & \dots & \dots & a_{n-1} & a_{n-2} \\ a_{2n-2} & a_{2n-3} & \dots & \dots & a_n & a_{n-1} \end{pmatrix}$$

Explain how an $n \times n$ Toeplitz matrix can be represented using $2n - 1$ elements. Then describe an algorithm that gets such a representation along with a vector v of length n , and outputs $A \cdot v$. Prove an upper bound on the running time of your algorithm. (15pts.)

(Note: A quadratic-time algorithm isn't good enough - design an algorithm running in time $O(n \cdot \log n)$.)

(Hint: To develop intuition, use notation for the entries of A as in the visual depiction, denote $v = v_0, v_1, \dots, v_{n-1}$, and write down the elements of $A \cdot v$ explicitly, then stare at them for a while. Now consider two polynomials p and q of degree at most $2n - 2$ such that the coefficients of p are $a_0, \dots, a_{2n-2}$ and the coefficients of q are $v_0, \dots, v_{n-1}, 0, \dots, 0$, and the entries of $A \cdot v$ are coefficients of the multiplication $p \cdot q$.)

Question 7. A person is climbing n stairs, and at each step they can climb either one stair or two stairs. The input to the problem is a sequence of real-valued costs $c_{1,1}, c_{1,2}, c_{2,1}, c_{2,2}, \dots, c_{n-2,1}, c_{n-2,2}, c_{n-1,1}$, where $c_{i,b}$ represents the cost of starting from stair $i \in [n]$ and climbing $b \in \{1, 2\}$ stairs, and costs may be negative. The total cost of a path from 1 to n is the sum of costs of steps along the way.

²When analyzing the hallucinated algorithm, you may assume for simplicity that n is a power of 5.

Write pseudocode for an algorithm that gets the sequence of costs and finds the minimal total cost of climbing n stairs. Prove the algorithm's correctness, and prove an upper-bound on its running time. (10pts.)

(Note: In this question and all other questions, if you are relying on dynamic programming and using a formula expressing the solution to a sub-problem in terms of solutions to other sub-problems, you must prove that your formula is correct.)

Question 8. A farmer has n plots of land, and in each plot they can either grow corn or grow pumpkins. They can also leave a plot empty. The only constraint is that no two consecutive plots i and $i + 1$ can grow the same crop.

The input to the problem is two sequences of positive real numbers $\{c_1, \dots, c_n\}$ and $\{p_1, \dots, p_n\}$ where c_i and p_i are the profits from planting corn and pumpkin at plot i , respectively. The total profit is the sum of profits in all plots. An empty plot yields zero profit.

Write pseudocode for an algorithm that gets two sequences of profits and finds the maximum total profit that the farmer can get. Prove the algorithm's correctness, and prove an upper-bound on its running time. (10pts.)

Question 9. In this problem the input is a string $x_1, \dots, x_n$ where $x_i \in \{a, b, c\}$ for each $i \in [n]$ (i.e., $x_1, \dots, x_n$ is a string of alphabet letters that can be either "a", "b", or "c"). For each consecutive substring $x_j, x_{j+1}, \dots, x_{j+k}$ (where $j \in [n]$ and $k \in 0, \dots, n - j$), define the discrepancy of the substring as the number of times "b" appears minus the number of times "a" appears; that is:

$$|\{m \in [j, j+k] : x_m = b\}| - |\{m \in [j, j+k] : x_m = a\}|$$

Write pseudocode for an algorithm that gets $x_1, \dots, x_n$ and finds the maximal discrepancy among all consecutive substrings. Prove the algorithm's correctness, and prove an upper-bound on its running time. (15pts.)

(Note: A quadratic-time algorithm isn't good enough – design a faster algorithm.)

Question 10. In this problem we are given $x_1, \dots, x_n, k \in \mathbb{Z}_d$. Our goal is to decide whether it is possible to insert $+$ and $-$ symbols between the numbers $x_1, \dots, x_n$ so that the resulting expression evaluates to k over $\mathbb{Z}_d$.³

Write pseudocode for an algorithm that gets $x_1, \dots, x_n, k \in \mathbb{Z}_d$, runs in time $O(n \cdot d)$, and outputs "yes" if it is possible to add $+$ and $-$ symbols so that the expression evaluates to k , and outputs "no" if it is impossible. Prove the algorithm's correctness, and prove the required upper-bound on its running time. (15pts.)

³Recall what it means to work over $\mathbb{Z}_d$: The elements $x_1, \dots, x_n, k$ are numbers in $\{0, \dots, d - 1\}$, and all arithmetic operations are performed modulo d .